



J. J. Strossmayer University of Osijek

Faculty of Electrical Engineering,
Computing and Information
Technologies Osijek
Kneza Trpimira 2b
HR-31000 Osijek
www.ferit.unios.hr

Lab Assignment 3:
Leader Election in Distributed Computing Systems

Academic year 2025/2026.

Zdravko Krpić, Ph.D, Associate Professor

Luka Loina, univ. mag. ing. comp.

When Amazon's "Leader" Went Missing

In March 2015, parts of Amazon Web Services (AWS) experienced a severe outage. Many applications relying on the cloud giant suddenly found themselves without access to vital resources. The root cause? A subtle bug in Amazon's internal coordination service.

The service used a leader election mechanism to decide which node was "in charge" of managing updates. When the elected leader unexpectedly failed, the remaining nodes should have quickly agreed on a new one. Instead, a flaw in the election logic caused multiple nodes to believe they were leader simultaneously, leading to conflicting updates and widespread instability.

For several hours, customers faced downtime that cost millions. The incident highlighted how fragile distributed systems can be when leader election goes wrong — too many leaders can be just as bad as none at all.

Today, frameworks like ZooKeeper and etcd, widely used in cloud and container platforms, include carefully tested leader election protocols to ensure that services continue smoothly even when failures occur.

Exercise 2: Leader Election in Distributed Computing Systems

1 Introduction

In distributed systems, processes often need to coordinate and agree on a single process to act as a *leader* or *coordinator*. The leader might be responsible for managing shared resources, handling client requests, or maintaining consistency among replicas. Because nodes can fail at any time, distributed systems must provide a reliable way to elect a leader dynamically.

This lab explores two classical leader election algorithms — the **Bully algorithm** and the **Ring algorithm** — implemented using MPI. You will learn how nodes can collectively agree on a coordinator, even in the presence of failures, and analyze the trade-offs of each approach.

In this lab, you will explore two classical leader election algorithms:

- **Bully Algorithm** — based on the idea that the process with the highest ID should take control when a failure is detected. Lower-ID processes may start an election, but they are eventually “bullied” by higher-ID processes until the strongest candidate wins.
- **Ring Algorithm** — assumes that processes are arranged in a logical ring. Election messages circulate around the ring, collecting IDs, until the highest-ID process is chosen and announced as the new leader.

Both algorithms solve the same problem but with different strategies and costs. While the Bully algorithm can converge quickly if the highest-ID process is alive, it may involve many messages. The Ring algorithm guarantees exactly one leader with a predictable message complexity, but may take longer in large systems.

In the exercises, you will implement both algorithms using MPI, experiment with process failures, and compare their behavior in terms of **message complexity, fault tolerance, and performance**.

1.1 Learning Objectives

By the end of this lab, you will be able to:

- Understand the role of leader election in distributed systems.
- Implement two classical leader election algorithms (Bully and Ring) in MPI.
- Compare their behavior in terms of message complexity and robustness.
- Relate these concepts to real-world distributed platforms like ZooKeeper, etcd, and Kubernetes.

2 Background

In distributed systems, processes often cooperate to provide a common service. Many services depend on the presence of a *coordinator* (or leader) that performs tasks such as:

- Managing access to shared resources.
- Acting as a central point for decision-making.
- Collecting and disseminating information.

When the leader fails or becomes unreachable, the system must elect a new one. The challenge is to guarantee that **exactly one leader** is chosen, even though processes operate concurrently and may fail at any time.

2.1 Requirements for Leader Election

A correct election algorithm must ensure:

1. **Safety** — at most one leader is elected at a time.
2. **Liveness** — eventually, a leader is elected (the algorithm does not deadlock).
3. **Fairness** — all processes should have a chance to participate.

2.2 Election Algorithms Covered in This Lab

- **Bully Algorithm**
 - Assumes each process has a unique identifier (ID).
 - If a process detects that the current leader has failed, it initiates an election by sending a message to all processes with higher IDs.
 - If none respond, it declares itself the leader.
 - If higher-ID processes respond, they continue the election.
 - Eventually, the process with the highest ID becomes the new leader and announces itself.
- **Ring Algorithm**
 - Processes are arranged in a logical ring.
 - An election is started by circulating an *ELECTION* message containing the initiator's ID.
 - Each process appends its own ID before forwarding.
 - When the message completes a full circle, the initiator determines the leader (highest ID) and announces it by sending a *COORDINATOR* message around the ring.

2.3 Complexity Considerations

- **Bully Algorithm:** In the worst case, the number of messages is $O(N^2)$.
- **Ring Algorithm:** Exactly $2N$ messages (one election, one announcement).

3 Guided Example: Bully Algorithm

The **Bully algorithm** is one of the earliest and simplest leader election protocols. Its central idea is straightforward:

- Each process has a unique identifier (ID).
- When a process suspects that the current coordinator has failed, it initiates an *election*.
- The process with the highest ID always wins and becomes the new leader.

In practice, the Bully algorithm assumes that:

1. Each process knows the IDs of all other processes.
2. Processes can reliably detect when the coordinator has failed (for this lab, you will simulate failure).
3. Message delivery is reliable and ordered.

3.1 Algorithm Outline

1. **Election Start**
 - a. A process (say rank i) notices that the current leader has failed.
 - b. It sends an *ELECTION* message to all processes with higher IDs ($j > i$).
2. **Response Phase**
 - a. If no higher-ID processes respond, i declares itself the leader.
 - b. If any higher-ID process responds, that process takes over the election.
3. **Coordinator Announcement**
 - a. Eventually, the highest-ID alive process wins.
 - b. It sends a *COORDINATOR* message to all other processes to announce itself as the new leader.

3.2 Step-by-Step Implementation (MPI)

We will now implement the Bully algorithm using MPI.

Step 1: Setup

- Start an MPI program with N processes.
- Assign each process an ID equal to its rank.
- Choose a coordinator initially (for example, the process with the highest rank).

- Store the ID of the current coordinator in a variable `leader`.

```
int rank, size;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

int leader = size - 1; // highest rank is initial coordinator
```

Step 2: Simulate a Failure

- Pretend that the current coordinator has failed.
- For example, if `rank == leader`, that process can simply skip participating (not send messages).
 - I.e. you should be able to “disable” the leader process.

Step 3: Start an Election

- Pick a process (e.g., rank 0) to detect the failure and start an election.
- It sends an *ELECTION* message (tag = 1) to all processes with higher rank.

```
if (rank == 0) { // initiator
    for (int j = rank + 1; j < size; j++) {
        MPI_Send(NULL, 0, MPI_INT, j, 1, MPI_COMM_WORLD); // tag=1 = ELECTION
    }
}
```

Step 4: Handle Election Messages

- When a process receives an *ELECTION* message, it replies with an *OK* (tag=2) if it is alive.
- It then starts its own election by forwarding *ELECTION* messages to processes with higher IDs.

```
MPI_Status status;
int flag;
MPI_Iprobe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &flag, &status);
if (flag) {
    if (status.MPI_TAG == 1) { // ELECTION
        MPI_Recv(NULL, 0, MPI_INT, status.MPI_SOURCE, 1, MPI_COMM_WORLD, &status);
        MPI_Send(NULL, 0, MPI_INT, status.MPI_SOURCE, 2, MPI_COMM_WORLD); // send OK
        // now forward election to higher ranks
        for (int j = rank + 1; j < size; j++) {
            MPI_Send(NULL, 0, MPI_INT, j, 1, MPI_COMM_WORLD);
        }
    }
}
```

Step 5: Declare Coordinator

- The highest-ID process that doesn't get any responses declares itself the leader.
- It then broadcasts a *COORDINATOR* message (tag=3) to all.

```
if (rank == size - 1) { // highest ID
    for (int j = 0; j < size; j++) {
        if (j != rank) {
            MPI_Send(&rank, 1, MPI_INT, j, 3, MPI_COMM_WORLD);
        }
    }
}
```

Step 6: Accept the New Leader

- When other processes receive a *COORDINATOR* message, they update their leader variable.

```

if (status.MPI_TAG == 3) { // COORDINATOR
    int new_leader;
    MPI_Recv(&new_leader, 1, MPI_INT, status.MPI_SOURCE, 3, MPI_COMM_WORLD, &status);
    leader = new_leader;
    printf("Process %d acknowledges leader %d\n", rank, leader);
}

```

3.3 Hints / Skeleton Code

You may use three message tags:

- 1 = ELECTION, 2 = OK, 3 = COORDINATOR.

Skeleton for probing/receiving:

```

MPI_Status status;
int flag;
MPI_Iprobe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &flag, &status);
if (flag) {
    if (status.MPI_TAG == 1) {
        // received ELECTION
    } else if (status.MPI_TAG == 2) {
        // received OK
    } else if (status.MPI_TAG == 3) {
        // received COORDINATOR
    }
}

```

Skeleton for sending:

```

MPI_Send(NULL, 0, MPI_INT, target, tag, MPI_COMM_WORLD);

```

3.4 What to Observe

- Run your implementation with 4–6 processes.
- Kill or simulate the coordinator's failure and observe how the election proceeds.
- Note the number of messages exchanged in different scenarios.
- Compare cases where the initiator has a **low ID** vs **high ID**.
- Compare to the Ring algorithm later: is Bully faster or slower in your setup?

3.5 Deliverable

By the end of this section, you should have a working Bully algorithm that:

1. Elects the highest-ID process as leader after a failure.
2. Correctly notifies all processes of the new leader.
3. Demonstrates message exchange and recovery from a failed coordinator.

4 Implementation: Ring Algorithm

The Ring algorithm provides an alternative approach to leader election.

Unlike the Bully algorithm, it requires no global knowledge of all process IDs — each process only communicates with its immediate successor in a logical ring.

Messages circulate until one process is recognized as the leader.

4.1 Goal

Implement the Ring leader-election algorithm in MPI so that:

- Any process can initiate an election.

- The highest-ID process is elected as the leader.
- All processes agree on the result after a coordinator announcement.

4.2 Implementation steps

Step 1. Create the Logical Ring

Each process determines its successor as:

```
int successor = (rank + 1) % size;
```

Each process keeps a variable `leader` (initially `size - 1`, i.e., the highest ID).

Print diagnostic information at startup:

```
Process 2 successor is 3
Process 3 successor is 0
```

Step 2. Start an Election

- Choose one process (e.g., rank 0) to simulate a coordinator failure and initiate an election.
- The initiator creates a message containing only its own ID (you may use an integer array or vector).
- Send the message to the successor with tag `ELECTION = 1`.

Hints:

```
MPI_Send(buffer, size, MPI_INT, successor, 1, MPI_COMM_WORLD);
```

Step 3. Circulate the Election Message

Each process that receives an *ELECTION* message should:

1. Insert its own ID into the message (in the first unused slot).
2. Forward the message to its successor.
3. Detect if the message has returned to the **initiator** (the process whose ID started the message).

If you are the initiator and the message returns, determine the **leader** — the highest ID contained in the list.

🔑 *Deliverable:* Verified message circulation (every process prints when it receives/forwards the message).

Step 4. Announce the Leader

- Once the initiator knows the leader, send a *COORDINATOR* message (tag 2) around the ring.
- Each process that receives the *COORDINATOR* message:
 1. Updates its local `leader` variable.
 2. Prints confirmation:

```
Process X acknowledges leader Y
```

3. Forwards the message to its successor.
- The initiator stops the algorithm when the message returns to it.

🔑 *Deliverable:* All processes output the same final leader ID.

4.3 Suggested MPI Tags

Message type	Tag value	Description
--------------	-----------	-------------

ELECTION	1	Circulating list of candidate IDs
COORDINATOR	2	Announcement of elected leader

4.4 Verification steps

After achieving a working implementation:

1. Run with 4–8 processes; observe message circulation order.
2. Start the election from different initiators (low-rank and high-rank).
3. Count the total number of sends — confirm it equals roughly $2 \times N$.
4. Discuss:
 - How predictable is message complexity?
 - What would happen if a process failed mid-election?

4.5 Final Deliverable

A complete MPI program that:

1. Arranges processes in a ring.
2. Conducts a leader election that selects the highest-ID process.
3. Circulates a coordinator message so all agree on the leader.
4. Prints final confirmation lines for every process.

5 Comparison and Experiments

Once you have completed the Bully and Ring algorithm implementations, it is time to evaluate, compare, and reflect on their behavior.

Both algorithms achieve the same goal — electing one leader among distributed processes — but they differ in how many messages they exchange, how fast they converge, and how they handle failures.

In this section, you will design and perform experiments, collect data, and analyze the results to better understand their trade-offs.

5.1 Goal

Your objective is to:

- Compare **message complexity**, **time to election**, and **fault tolerance behavior** of the Bully and Ring algorithms.
- Provide a short, evidence-based summary of your findings.
- Develop intuition for when each algorithm might be preferable in real distributed systems.

5.2 Experimental Setup

Before starting:

1. Reuse the same MPI environment for both algorithms.
2. Use at least **4–8 processes**.
3. Ensure both programs print enough information to trace elections (e.g., message sending, new leader announcement).
4. Run all experiments multiple times for consistency.

You are encouraged to collect results in a **CSV table** or **spreadsheet** for later discussion.

5.3 Experimental Procedure

Step 1. Message Counting

1. Modify your implementation to count the number of MPI send operations used during the election.
 - You can maintain a local counter (`int msg_count++` each time you call `MPI_Send`).
 - After the election, gather counts from all processes using `MPI_Reduce` or `MPI_Gather`.
2. Record the total number of messages sent.

📁 *Deliverable:* A small table such as:

N (processes)	Bully messages	Ring messages
4		
6		
8		

Step 2. Time to Election

1. Measure the wall-clock time for election process using `MPI_Wtime()`.
2. Place the first timestamp immediately before the initiator starts the election, and the second after all processes have acknowledged the leader.
3. Record average election times over at least 3 runs per algorithm.

📁 *Deliverable:* A comparison chart (text or plot) of election duration versus number of processes.

Step 3. Failure Scenarios

Simulate different failures to explore robustness.

- **Scenario A:** Leader fails (highest-rank process disabled).
- **Scenario B:** A middle-rank process fails during the election process.
- **Scenario C:** The initiator of the election fails after starting it.

For each scenario:

- Modify your implementation to simulate failure scenario.
- Observe whether a new leader is eventually elected and how long it takes.
- Note any cases where the election fails or deadlocks.

📁 *Deliverable:* A short paragraph summarizing each scenario and observed behavior.

5.4 Hints for Measurements

- Use `MPI_Barrier(MPI_COMM_WORLD)` to synchronize all processes before timing.
- Print only summary results, not every message, during performance runs to avoid console overhead.
- To simulate failure, simply skip `MPI_Send`/`MPI_Recv` in the “failed” process.
- Optionally, add random delays (`sleep()` or `usleep()`) to simulate asynchronous behavior.

5.5 Student Tasks

Assignment 1. Bullys algorithm (mandatory)

1. Implement Bullys Algorithm by following steps in 3. Guided Example.
2. Do experimental analysis of Bullys Algorithm by following steps described in Experimental Procedure.

Assignment 2. Ring algorithm

1. Implement Ring algorithm by following steps in 4. Implementation: Ring Algorithm.
2. Do experimental analysis of Ring Algorithm by following steps described in Experimental Procedure.

Assignment 3. Qualitative Comparison (BONUS)

Discuss the following questions in your report:

1. **Message Overhead:** Which algorithm exchanges more messages for the same N ? Why?
2. **Speed of Convergence:** Which algorithm tends to finish faster in your experiments?
3. **Fairness and Determinism:** Does every process have an equal chance to become leader?
4. **Scalability:** What happens as N grows larger (e.g., 10, 20 processes)?
5. **Fault Tolerance:** Which algorithm handles failures more gracefully?

5.6 Optional Extension: Visualization

If you want to go further, visualize the message exchange:

- Log (rank, tag, timestamp) for each send/receive into a text file.
- Plot message flow using Python/Matplotlib or Graphviz. This gives insight into message ordering and delays.

5.7 Final Deliverables

By the end of this section, you should have:

1. **Measured data** on messages and time for both algorithms.
2. **Written analysis** of strengths and weaknesses.
3. **Working MPI programs** demonstrating both elections.
4. (Optional) **Charts or visualizations** summarizing the results.

Your comparison should conclude **which algorithm performs better and under what conditions**, linking your observations to theoretical complexity ($O(N^2)$ for Bully vs $O(N)$ for Ring).